



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Marin Kišić, Josip Klepec, Dorijan Lendvaj, Martina Licul, Patrick Pavić, and Petar Sruk.

Implementation examples are given in attached source code files.

Task Kalendar

Prepared by: Marin Kišić and Martina Licul

Necessary skills: loops, strings, arrays

We can create an array consisting of $d - 1$ zeroes, the integers from 1 to n , and then, if necessary, some more zeroes, so that the length of the array is divisible by 7. The zeroes represents the missing days, i.e. the ones we will replace with '... '.

The length of this array divided by 7 is the number of rows the calendar will have. We can now iterate through the array and create the rows: depending on the element of the arrays, we will determine which three characters should we add.

The above described solution is without decorations. We can add them easily: the first and last row are always the same, and all the other rows can be made with a little addition to the solution from above.

Task Diskurs

Prepared by: Josip Klepec

Necessary skills: bitmasks

Let's look at the numbers as bitstrings of length m . For a number x , let x^c be a number that has *flipped* binary representation.

For example, for $m = 5$ and $x = 13$ it follows $x^c = 18$ because $x = 01101_2$ and $x^c = 10010_2$. Exactly x^c would be ideal candidate for biggest *hamming distance* with x , but x^c is not necessarily a part of a .

Let us describe two possible solutions:

- First solution is based on the fact that

$$\max_{y \in a} \text{hamming}(x, y) = m - \min_{y \in a} (x^c, y).$$

intuitively, we want a number as similar to x^c as possible.

Let us try to calculate minimal *hamming distance* for every number in $[0, 2^m - 1]$. For every number that appears in a it is 0. It leads us to a BFS solution, where each number in $[0, 2^m - 1]$ is a node in our graph and the edges represent changing exactly one bit. In a graph constructed in such fashion *hamming distance* between two numbers is exactly the length of shortest path between corresponding nodes. So we start BFS algorithm with every number from a in our queue and we will get the desired minimal distances.

- Second solution is based on dynamic programming. Let $\text{popcount}(x)$ number of ones in binary representation of x . Solution will be based on following identity:

$$\text{hamming}(x, y) = \text{popcount}(x) + \text{popcount}(y) - 2 \cdot \text{popcount}(x \& y)$$

Intuitively, each one in binary representation of each number contributes 1 to *hamming distance*, but then each common one decreases it by 2.

We want to calculate $\text{dp}(\text{mask})$ - maximum number of ones for some number in a but each one that is not in mask we penalize by 2. Formally,

$$\text{dp}(\text{mask}) = \max_{x \in a} (\text{popcount}(x) - 2 \cdot \text{popcount}(x \& \text{mask}^c))$$



Then, the solution for some x will be given as $\text{popcount}(x) + dp(x^c)$. The above dp can be calculated with standard dp with bitmasks methods. For details check the official solution where this dp is called *dpDown*.

Complexity of both solutions is $\mathcal{O}(2^m \cdot m)$.

Task Logaritam

Prepared by: Petar Sruk

Necessary skills: prime factorization, math

Since $a_{1..1} = a_1 + a_1$, a_1 has to be equal to 0, so if the first element is changed the sequence cannot become logarithmic again. It is easily shown that $a_{xy} = a_x + a_y$ for every pair of positive integers x, y is equivalent to $a_m = \alpha_1 a_{p_1} + \alpha_2 a_{p_2} + \dots + \alpha_k a_{p_k}$ for positive integer $m = p_1^{\alpha_1} \dots p_k^{\alpha_k}$. We conclude that logarithmic sequence is uniquely defined by values at prime indices. If the index p of changed element is prime one possible way to "fix" the sequence is to change all of the elements at indices which are multiples of p , there are $\lfloor \frac{n}{p} \rfloor - 1$ of them. If that is the minimal number of changes necessary, then for any index x of changed element the minimal number of changes necessary will be $\lfloor \frac{n}{\text{greatest prime factor of } x} \rfloor - 1$ (since it is necessary to change at least one prime factor of x).

We will prove that is the solution and we can implement it with standard algorithm for finding prime factorization in $\mathcal{O}(q\sqrt{n})$.

Proof: Using induction by n let's prove a stronger claim for odd primes p : "Changing multiples of p is the only minimal solution".

If n is not divisible by p the number of changes stays the same and the solution is unique.

If n is dividisible by p then clearly the solution for $n - 1$ needs to be expanded by changing n -th element and we get that $\lfloor \frac{n}{p} \rfloor - 1$ is the least number of changes. Assuming that there exists another minimal solution that solution doesn't change n -th element and it is necessary to change another odd prime index q (for $q = 2$ it can be seperately shown it requires more than minimal changes) for which by induction hypothesis has unique solution for first $n - 1$ element. Since $2q$ is not divisible by p that solution would have at least $\lfloor \frac{n}{p} \rfloor$ changes so it is not minimal.

For $p = 2$ it doesn't necessarily hold that minimal solution is unique (for prime $\frac{n}{3} < q \leq \frac{n}{2}$ it is possible to change q instead of $2q$), however it still holds that the minimal number of changes necessary is $\lfloor \frac{n}{2} \rfloor - 1$. The proof similar to the previous is left for the reader as exercise as well as finding implementation which runs in time complexity of $\mathcal{O}(n + q)$.

Task Slastičarnica

Prepared by: Vito Anić and Patrick Pavić

Necessary skills: dynammic programming

Let $dp(l, r)$ be the maximum number of customers we can serve if we still have cakes $a_l, a_{l+1}, \dots, a_r$ at the table. For the first subtask, we can use the simple transition:

$$dp(l, r) = \max(\max_{i \leq l}^{i=0} (dp(i, r) + \text{check}(i, l - 1, dp(i, r))), \max_{j \leq r}^{j=r+1} (dp(l, j) + \text{check}(r + 1, j, dp(l, j))),$$

where $\text{check}(L, R, x)$ is a function that returns 1 if it is possible to satisfy customer x with the cakes $a_L, a_{L+1}, \dots, a_R$, and 0 otherwise. Time complexity is $\mathcal{O}(n^3)$ or $\mathcal{O}(n^4)$, depending on the implementation of the function *check*.

Similarly, for the second subtask if all d_i are equal to 1, then we only need to check $dp(l - 1, r)$ and $dp(l, r + 1)$ to determine $dp(l, r)$.

That leads us to the complete solution. Is it possible somehow to check only neighboring states and get the required solution? It is! Instead of memorizing only the maximum number of customers we can serve,



we will memorize two more things: the biggest number of cakes that is possible to give to the current customer outside the left and the right border. For example, let k be the number of cakes we can give to the current customer outside of the left border. Two things need to be satisfied:

- $dp[l - k][r] = x$ (it is possible to give all the cakes to the current customer)
- $a_{l-k}, a_{l-k+1}, \dots, a_{l-1} \geq s_x$ (all the cakes have sweetness greater or equal to the required value)

Now we do not need to check every state, only the neighbouring ones. When one of the counters becomes equal to d_x , we can give those cakes to the current customer, and increase the customer number by 1. Total time complexity is $\mathcal{O}(n^2)$.

Task Zastave

Prepared by: Patrick Pavić

Necessary skills: calculus, binary search

Let's look at the case $n = 1$. Let $f(x)$ be the area of the flag with height x . It follows $f(x) = x\sqrt{r^2 - x^2}$. The function is increasing until $x = \frac{r}{\sqrt{2}}$ and decreasing afterwards. It follows for $x < \frac{r}{\sqrt{2}}$ the solution is $f(x)$ and for $x \geq \frac{r}{\sqrt{2}}$ the solution is $\frac{r^2}{4}$.

Notice:

$$f'(x) = \sqrt{r^2 - x^2} - \frac{x^2}{\sqrt{r^2 - x^2}}$$

Now for the general case. The function $f(x, r)$ will be analogous to the previous but for any r . Notice that the function $f'(x)$ is decreasing on the interval $(0, \frac{r}{\sqrt{2}})$. If we were solving the discrete version, it would be enough to always take the element which would increase our solution the most, the correctness of this algorithm relies on the fact that the discrete derivative is decreasing. In the continuous case, the analogous solution is to always move the point which currently has the largest derivative. But, it is easier to fix a derivative t and check whether the sum of x_i 's such that $f'(x_i, r_i) = t$ is smaller than S . Notice that for such x_i :

$$x_i = \sqrt{\frac{4r_i^2 - t^2 - \sqrt{(t^2 - 4r_i^2)^2 - 16(r_i^4 - t^2r_i^2)}}{8}}$$

Now it is enough to find such a t with binary search, it is enough to do 100 steps for the wanted precision. The final complexity is $\mathcal{O}(100n)$. There are some edge cases that need to be checked by hand.